

PHP y SQL

Álvaro González Sotillo

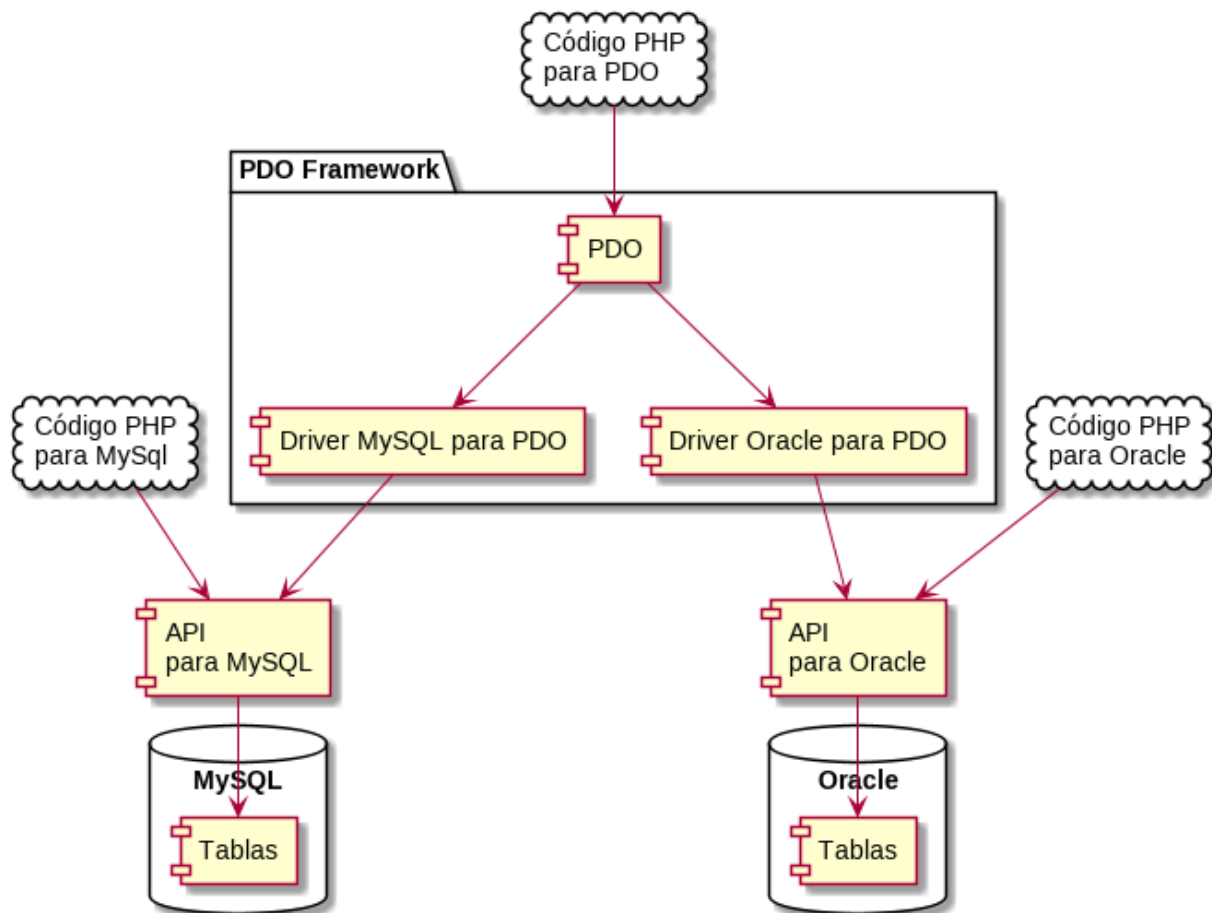
5 de diciembre de 2024

Índice

1. API, <i>driver</i> y PDO	1
2. <code>mysqli</code> : API orientada a objetos o procedimental	2
3. Conexión	2
4. Sentencias	3
5. Transacciones	3
6. Referencias	3

1. API, *driver* y PDO

- *API*: conjunto de funciones/objetos que permiten conectar a una base de datos SQL
- PDO: conjunto de funciones que pueden acceder a cualquier base de datos, siempre que haya un *driver* para ello
 - El *driver* adapta una API al PDO
- Por ser más simple, utilizaremos el **API `mysqli`**



2. **mysqli**: API orientada a objetos o procedimental

- **mysqli** puede usarse de dos formas
 - Como métodos de un objeto conexión
 - Como funciones que reciben una conexión
- No se recomienda mezclarlas

```
// ORIENTADO A OBJETOS
$conexion = new mysqli("ejemplo.com", "usuario", "contraseña", "basedatos");
$conexion->query("DROP TABLE IF EXISTS test");

// PROCEDIMENTAL
$conexion = mysqli_connect("ejemplo.com", "usuario", "contraseña", "basedatos");
mysqli_query($conexion, "DROP TABLE IF EXISTS test");
```

3. **Conexión**

- **mysqli_connect**
 - Utiliza opciones por defecto
 - Recibe servidor, usuario, contraseña, base de datos y puerto.
- **mysqli_init -> mysqli_options -> mysqli_real_connect**
 - Permite la configuración de varios parámetros, como el *timeout* de la conexión.

4. Sentencias

- Cuando se hace una consulta, el servidor lee los ficheros necesarios hasta tener las primeras filas
 1. El cliente puede requerir todos los datos de una vez, y libera al servidor. Esto puede sobrecargar la memoria del cliente.
 2. El cliente puede ir pidiendo datos según necesita. El servidor debe mantener datos en memoria.

4.1. Todo en memoria del cliente

```
$resultado = $mysqli->query("SELECT id FROM test ORDER BY id ASC");

echo "Orden del conjunto de resultados...\n";
$resultado->data_seek(0);
while ($fila = $resultado->fetch_assoc()) {
    echo " id = " . $fila['id'] . "\n";
}
```

4.2. Como *stream*, sin liberar al servidor

```
$mysqli->real_query("SELECT id FROM test ORDER BY id ASC");
$resultado = $mysqli->use_result();

echo "Orden del conjunto de resultados...\n";
while ($fila = $resultado->fetch_assoc()) {
    echo " id = " . $fila['id'] . "\n";
}
```

<https://www.php.net/manual/es/mysqli.summary.php>

4.3. Sentencias preparadas

- Las sentencias suelen ser cadenas dinámicas
 - Los datos a seleccionar/borrar/actualizar dependen de parámetros
- Existen la tentación de construir *strings* con las sentencias

```
$id = $_GET["id"];
$mysqli->query("delete from USUARIOS where id='$id'");
```

¿Qué ocurre si id es ' or '1'='1' -- ?

5. Transacciones

```
$mysqli = new mysqli("ejemplo.com", "usuario", "contraseña", "basedatos");
$mysqli->autocommit(false);

$mysqli->query("INSERT INTO test(id) VALUES (1)");
$mysqli->rollback();

$mysqli->query("INSERT INTO test(id) VALUES (2)");
$mysqli->commit();
```

6. Referencias

- Formatos:
 - [Transparencias](#)
 - [PDF](#)
 - [Página web](#)
 - [EPUB](#)

-
- Creado con:
 - [Emacs](#)
 - [org-re-reveal](#)
 - [Latex](#)
 - Alojado en [Github](#)